

Stacks and Santa Barbara Weather Puzzle

A stack manages keys using the principle of Last in First Out (LIFO) via four operations, each $O(1)$: 1. `push(value)` 2. `pop()` 3. `top()` 4. `empty()`

A stack is useful for keeping track of history information where computation only depends on the most recent information !!

Objective: Analyze the complexity of a naive solution for the Daily Temperatures problem by determining its complexity. Next, optimize using a stack-based solution. Explore time and space complexity tradeoffs.

Problem: Given an array of daily temperatures, return an array `answer` where `answer[i]` is the number of days after day `i` until a *warmer* temperature occurs, or 0 if none exists. Use Santa Barbara's forecast:

Results for **Santa Barbara, CA** · [Choose area](#) ⋮



75

°F | °C

Precipitation: 5%
Humidity: 76%
Wind: 5 mph

Weather

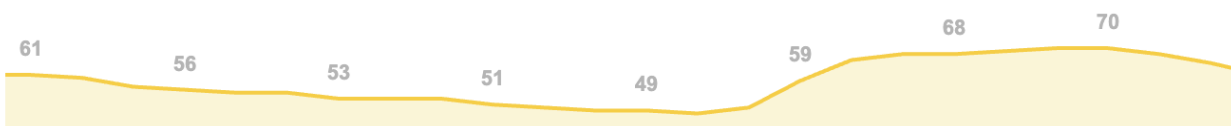
Sunday

Partly sunny

Temperature

Precipitation

Wind



7 PM	10 PM	1 AM	4 AM	7 AM	10 AM	1 PM	4 PM
Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun
75° 48°	70° 45°	75° 45°	80° 46°	77° 46°	70° 46°	71° 47°	72° 49°

Temp:
Day:

75 70 75 80 77 70 71 72
0 1 2 3 4 5 6 7
→ ↑

Leetcode (medium) daily temperatures:

<https://leetcode.com/problems/daily-temperatures/>

Ans: 3 0 0

Part 1: Naive Solution - Turn the problem's definition into an algorithm. (5 mins)

Approach: For each i , look to future days to find a warmer day

Write the pseudo code of your algorithm

None

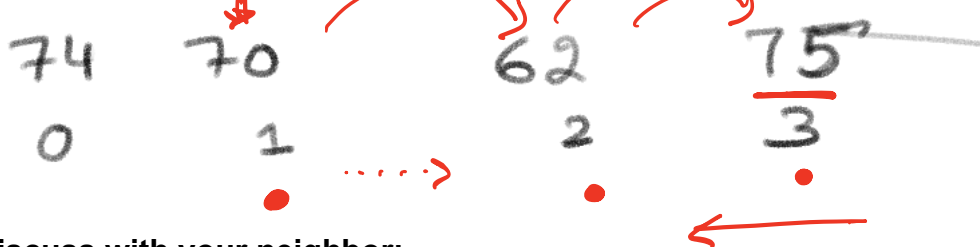
Run your algorithm on the given input temp data

Temp:	75	70	75	80	77	70	71	72
Day:	0	1	2	3	4	5	6	7

Ans: [_____, _____, _____, _____, _____, _____, _____, _____]

Part 2: Complexity Analysis of Naive Solution (Discuss with your peer)

1. How many comparisons did you make to get the answer for day 7 for the given input? _____
2. How many comparisons did you make to get the answer for day 0 for the given input? _____
3. Write an 8-day temperature input vector that incurs the worst-case running time
[_____, _____, _____, _____, _____, _____, _____, _____]
4. What is the worst-case time and space complexity expressed in Big-O



(5 mins) Discuss with your neighbor:

What is the redundant work being done in your brute-force solution?

Could a helper data structure avoid the redundant work? Write your key insight.

Analyze the brute-force approach.

We scan right to left. At each day, we want to know if there is a warmer day to my right

$i = 3$, nothing to my right. $ans[3] = 0$

$i = 2$, look to the right. see 75. 75 is warmer than 62. $ans[2] = 1$

$i = 1$, " " " " see 62, 75. " " " 70 $ans[1] = 2$

74 70 62 75
0 1 2 3

Remember values in a stack

0 (74)
~~1 (70)~~
~~2 (62)~~
3 (75)

$i = 3$ check 75 ✓ $ans[3] = 0$
 $i = 2$ check day 2 (62) $ans[2] = 1$
 check top of stack $ans[1] = 2$
 compute answer $ans[0] = 3$
 $i = 1$ check day 1 (70)
 check top of stack (62)
 Since 62 < 70
 pop()
 compute answer
 $i = 0$

Part 3: Stack-Based Solution

Key insight: Each day could be the answer for a preceding day: remember it in a stack
When a warmer day arrives, all cooler days behind it become irrelevant: pop them.

Trace the stack for the example below

Input: temperature on each day:

Temp	65	60	62	70	68	64	66	72
Day	0	1	2	3	4	5	6	7

Ans: Num days until a warmer day:

Ans	3	1	1	4	3	1	1	0
Day	0	1	2	3	4	5	6	7

Show the state of the stack after the temperature for each day is processed!

Stack								
Top								
				4(60)				0(65)
			5(64)	5(64)	3(70)	2(62)	1(60)	1(60)
		6(66)	6(66)	6(66)	4(68)	3(70)	2(62)	2(62)
	7(72)	7(72)	7(72)	7(72)	7(72)	7(72)	7(72)	7(72)
Bottom								

Note:
Stack
is
mono-
tonically
decreasing

Iteration 0 1 2 3 4 5 6 7

The visualization above shows how the stack evolves with every iteration. Provides a trace of the algorithm I coded in class.

Key insight → We never pop more than we push, one push per iteration ⇒ Total pop = 0(n)

1. Complete the code to capture the stack-based solution from the previous page as the input vector is traversed right to left:

```
#include <vector>
#include <stack>

std::vector<int> dailyTemperatures(std::vector<int>& temperatures) {
    int n = temperatures.size();
    std::vector<int> answer(n, 0);
    std::stack<int> stack;
    //complete this code

    return answer;
}
```

2. What is the time and space complexity of this solution?

Next steps:

- Attempt a different solution to this problem, traversing the input vector left to right.
- Discuss your solutions with the course staff in office hours